

Assignment 1 (100 points)

Task 1: (30 points)

Part 1 – Data Structure Selection Reasoning

You are given the following scenarios. For each, **choose the most appropriate Java data structure** and **justify your choice** in 3–5 sentences.

1. **Recent Actions in a Text Editor** – You need to store and undo the most recent operations performed by the user.
2. **Customer Service Call Center** – Customers are served in the order they call.
3. **Leaderboard in a Game** – Players are ranked by their score, highest first.
4. **Unique Email Addresses** – You need to store a collection of email addresses without duplicates.
5. **Student Grades Lookup** – You need to quickly find a student's grade given their student ID.
6. **Playlist of Songs** – You need to store songs in order and allow random access by index.

Part 2 – OOP Concept Identification

Below is a short Java code snippet. **Identify which OOP concept(s)** are demonstrated in each marked section and explain your reasoning.

Tasks:

1. For [A], identify the OOP concept(s) and explain why.
2. For [B], identify the OOP concept(s) and explain why.
3. For [C], identify the OOP concept(s) and explain why.

Write a short paragraph (5–7 sentences) answering:

- Why is it important to choose the right data structure for a problem?
- How do OOP concepts help in making code more maintainable and reusable?

```

// [A]
public abstract class Shape {
    private String color;

    public Shape(String color) {
        this.color = color;
    }

    public String getColor() { return color; }
    public void setColor(String color) { this.color
= color; }

    public abstract double area();
}

// [B]
public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}

// [C]
public class ShapePrinter {
    public void printArea(Shape s) {
        System.out.println("Area: " + s.area());
    }
}

```

Task 2 (30 points)

Part 1:

Below are five scenarios. For each, you must:

1. Identify the most likely **time complexity** (choose from $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$).
2. Justify your choice in **3–5 sentences**, explaining the reasoning and trade-offs.

Scenarios:

1. Looking up a value in a **HashMap** by key.
2. Performing a **binary search** on a sorted array.
3. Printing all elements in a list of size n.
4. Sorting an array using **Merge Sort**.
5. Checking if a list contains any **duplicate elements** using a nested loop.

Deliverable: A short-written paragraph with:

- Complexity choice for each scenario.
- Justification for time complexity.

Part 2:

Problem Statement: Write a Java program that:

1. Reads an integer n from the user.
2. Generates an array of n random integers.
3. Finds and prints:
 - The maximum value in the array.
 - The sum of all elements.
 - Whether the array contains any duplicates.

Requirements:

- Implement each of the three operations in separate methods.
- After writing the code, analyze each method:
 - State its time complexity
 - Explain why.

Task 3 (40 points)

Part 1:

Instructions: You are given three sorting techniques and two searching techniques:

- $O(n^2)$ Sorts: Bubble Sort, Insertion Sort
- Non- $O(n^2)$ Sorts: Merge Sort (recursive), Quick Sort (recursive)
- Search Techniques: Linear Search, Binary Search (recursive)

For each technique:

1. Describe how it works in your own words (max 150 words each).
2. Write the pseudo-code (language-agnostic, but clear and structured).
3. Calculate and discuss performance:
 - Best-case, worst-case, and average-case time complexity.
 - Space complexity (including recursion stack if applicable).
4. For Merge Sort, Quick Sort, and Binary Search:
 - Clearly identify the divide and conquer steps.
 - Explain the base case and recursive case.

Deliverable: A well-structured document or table summarizing:

- Algorithm name
- Description
- Pseudo-code
- Time complexities

Part 2:

Problem Statement: Create your own list of 10,000 random integers. Your task is to:

1. **Implement:**
 - Bubble Sort ($O(n^2)$)
 - Merge Sort (recursive, $O(n \log n)$)
 - Binary Search (recursive)

2. Measure and compare:

- Execution time for Bubble Sort vs Merge Sort on the same dataset.
- Execution time for searching a value using Linear Search vs Binary Search (on a sorted list).

3. Demonstrate the following Collections class methods:

- `Collections.sort()` – Sort the list and compare performance with your Merge Sort.
- `Collections.binarySearch()` – Search for a value and compare with your Binary Search.
- `Collections.reverse()` – Reverse the sorted list.
- `Collections.shuffle()` – Randomize the list order.

Requirements:

- Use `System.nanoTime()` or `System.currentTimeMillis()` for timing.
- Print results in a comparison table.
- Explain why the performance differences occur.